

Laboratorul 6

Lucrul în rețea

1 Retele de calculatoare

O rețea de calculatoare presupune conectarea fizică a mai multor calculatoare numite și *noduri* (prin analogie cu grafurile, rețelele pot fi modelate și analizate teoretic folosind teoria grafurilor) sau *host-uri* cu ajutorul unor medii de comunicare. Comunicarea efectivă dintre calculatoare folosind aceste medii de comunicare presupune două premize fundamentale: calculatoarele implicate în comunicare trebuie să se poată identifica unul pe altul și respectiv trebuie să se poată ”înțelege” unul cu altul. Prima premiză este asigurată prin asignarea unei adrese de rețea fiecărui calculator. Cea de-a doua este realizată cu ajutorul protocoalelor de comunicație. În mod uzual, acestea determină și modul de adresare.

În cazul Internetului, protocoalele de comunicație sunt denumite generic TCP/IP, deși în fapt e vorba de o suită de protocoale. Adresele asignate calculatoarelor se numesc adrese IP (Internet Protocol) și sunt folosite de protocoalele de comunicație. O adresă de IP este compusă din 4 octeți separați de punct, de exemplu 192.168.0.1. Odată identificat, un calculator poate oferi o multitudine de servicii: web, mail, transfer de fișiere, etc. TCP/IP identifică aceste servicii prin porturi cu numere: 80 pentru web, 25 pentru mail, 21 pentru transfer de fișiere, șamd. Ele se numesc *well-known ports* pentru că sunt public cunoscute de toată lumea (ca într-o carte de telefoane). O listă de well-known ports în sistemele Unix se poate găsi în fișierul `/etc/services`.

2 Domenii

Deși stau la baza comunicării în internet, adresele de IP sunt mai rar folosite direct, calculatoarele având în general un nume lizibil, e.g. `www.google.com`, care este asociat cu adresa lor de IP. Servere specializate numite **Domain Name Servers** (DNS), accesibile printr-un protocol de comunicație special disponibil ca serviciu pe portul 53, răspund cererilor pe care alte calculatoare conectate la

internet, uzual definite ca fiind calculatoare client (sau pe scurt, clienti), le fac pentru a afla fie numele unui calculator data fiind adresa sa de IP, fie adresa de IP a unui calculator cunoscut dupa numele sau.

Concret, pentru a obține adresa de IP asociata unui host putem folosi mai multe metode. Comanda **nslookup(1)** (*name server look-up*) primește ca prim argument numele *host*-ului, asa-numitul Fully Qualified Domain Name sau FDQN, și, opțional, un al doilea argument care specifică ce server DNS să folosească pentru a căuta informația.

```
$ nslookup fmi.unibuc.ro
Server:          213.154.124.1
Address:         213.154.124.1#53
```

```
Non-authoritative answer:
Name:    fmi.unibuc.ro
Address: 193.226.51.6
```

În prima parte sunt afișate date legate de server-ul DNS folosit. A doua parte oferă informațiile cerute: numele și adresa. Dacă dorim să întrebăm un server anume (în exemplul de mai jos server-ul Google) îi punem adresa în al doilea argument:

```
$ nslookup fmi.unibuc.ro 8.8.8.8
Server:          8.8.8.8
Address:         8.8.8.8#53
```

```
Non-authoritative answer:
Name:    fmi.unibuc.ro
Address: 193.226.51.6
```

Comanda **nslookup** poate fi lansata si in mod interactiv, caz in care ofera o mica linie de comanda care permite executia anumitor instructiuni, dupa cum se poate vedea mai jos:

```
$ nslookup
> server
Default server: 127.0.1.1
Address: 127.0.1.1#53
> server 1.1.1.1
Default server: 1.1.1.1
Address: 1.1.1.1#53
> set type=ptr
> 8.8.8.8
Server:          1.1.1.1
Address:         1.1.1.1#53
```

```
Non-authoritative answer:
8.8.8.8.in-addr.arpa    name = dns.google.
```

Authoritative answers can be found from:

```
> set type=a
> dns.google
Server:      1.1.1.1
Address:     1.1.1.1#53
```

Non-authoritative answer:

```
Name:  dns.google
Address: 8.8.4.4
Name:  dns.google
Address: 8.8.8.8
```

```
> set type=ns
> google.com
Server:      1.1.1.1
Address:     1.1.1.1#53
```

Non-authoritative answer:

```
google.com      nameserver = ns1.google.com.
google.com      nameserver = ns2.google.com.
google.com      nameserver = ns3.google.com.
google.com      nameserver = ns4.google.com.
```

Authoritative answers can be found from:

```
> set type=mx
> google.com
Server:      1.1.1.1
Address:     1.1.1.1#53
```

Non-authoritative answer:

```
google.com      mail exchanger = 10 smtp.google.com.
```

Authoritative answers can be found from:

```
> exit
```

Comanda *server* tipareste numele serverului la care apeleaza **nslookup** momentan pentru a rezolva cereri de DNS. Daca primeste ca parametru o adresa de IP sau un FDQN (Fully Qualified Domanin Name) a unui server DNS, va schimba adresa serverului DNS la care *nslookup* apeleaza. Comanda *set* in conjunctie cu parametrul *type* permite efectuarea de query-uri de FDQN (*type A*) care returneaza adresa IP corespunzatoare, query-uri de adrese IP (*type PTR*) care returneaza FDQN-ul corespunzator, query-uri pentru a afla serverul de DNS responsabil pt un anumit domeniu (*type NS*) sau pentru a afla serverul de mail (*type MX*) responsabil pentru un anumit domeniu.

Alte comenzi utile care functioneaza similar sunt:

- **dig(1):** \$ **dig @8.8.8.8 fmi.unibuc.ro** – server-ul DNS trebuie pref-

xat cu @

- **host(1):** \$ **host fmi.unibuc.ro 8.8.8.8** – server-ul DNS apare la sfarsitul comenzii
- **whois(1):** \$ **whois unibuc.ro** – informații despre domeniul principal, nu despre subdomeniul fmi)

Pentru a vedea dacă un *host* este accesibil în rețea putem folosi comanda **ping(1)**. Faceti distincția între conectare și accesibilitate. Un calculator poate fi fizic conectat la rețea, dar temporar inaccesibil, fie din cauza unor defecte (hardware sau software) locale pe *host* fie din cauza unor defecte în rețeaua din care face parte. De asemenea, se întâmplă adesea ca un *host* să fie temporar inaccesibil pentru mentenanță. **ping** nu spune decât dacă la momentul executiei comenzii *host*-ul este accesibil (se mai spune și *online*) sau nu.

```
$ ping fmi.unibuc.ro
PING fmi.unibuc.ro (193.226.51.6): 56 data bytes
64 bytes from 193.226.51.6: icmp_seq=0 ttl=50 time=7.317 ms
64 bytes from 193.226.51.6: icmp_seq=1 ttl=50 time=7.053 ms
64 bytes from 193.226.51.6: icmp_seq=2 ttl=50 time=6.925 ms
^C
— fmi.unibuc.ro ping statistics —
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/std-dev = 6.925/7.098/7.317/0.163 ms
```

Observați că această comandă întâi găsește adresa *host*-ului și apoi comunică direct cu IP-ul acestuia. În Unix, dacă nu se specifică un număr de încercări, comanda va încerca până când utilizatorul o oprește cu **Ctrl-C**. În Windows, comanda încearcă de 4 ori implicit după care se oprește.

O altă comandă utilă atunci când vreți să obțineți mai multe informații despre accesibilitatea unui nod de rețea din internet este **traceroute**. De pildă, dacă vreți să știți peste câte *hop-uri* trece un pachet trimis de mașina locală ca mai sus în comanda **nslookup** către serverul DNS public al Google, puteți folosi comanda de mai jos:

```
$ traceroute 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1 gateway (192.168.100.1) 0.837 ms 0.809 ms 7.898 ms
 2 10.0.33.143 (10.0.33.143) 5.829 ms 5.796 ms 5.880 ms
 3 10.12.100.1 (10.12.100.1) 5.839 ms 5.912 ms 5.871 ms
 4 10.220.187.54 (10.220.187.54) 5.905 ms 5.869 ms 5.931 ms
 5 10.220.208.244 (10.220.208.244) 19.839 ms 20.122 ms 45.225 ms
 6 72.14.216.212 (72.14.216.212) 25.055 ms 209.85.168.182 (209.85.168.182)
18.447 ms 16.041 ms
 7 * * 172.253.65.251 (172.253.65.251) 21.788 ms
 8 dns.google (8.8.8.8) 19.253 ms 142.251.65.217 (142.251.65.217)
22.249 ms 209.85.244.147 (209.85.244.147) 19.804 ms
```

După cum observați, în cazul calculatorului de pe care a pornit comanda a fost nevoie să se treacă de 7 routere până să se ajungă la destinație (*dns.google*). Aceste routere intermediare se numesc în limbaj colocvial *hop-uri*.

Fișierul `/etc/hosts` este folosit pentru a defini manual perechi adresă IP – nume. În momentul în care se fac query-uri de DNS, clientul local DNS de pe calculator (asa numitul *DNS resolver*) cercetează fișierul `/etc/nsswitch.conf` pentru a detecta ordinea în care se caută un server care să rezolve cererea. În general, câmpul `hosts` al fișierului `/etc/nsswitch.conf` precizează ordinea de mai jos:

```
hosts:                files dns
```

Această sintaxă ne spune că în general fișierele locale (*files*), în cazul nostru `/etc/hosts`, sunt chestionate prioritar pentru rezolvarea numelui înaintea serverelor DNS. În cazul în care fișierul `/etc/hosts` conține rezolvarea dorită a numelui, DNS resolver-ul întoarce rezultatul găsit. Dacă nu a găsit nicio intrare potrivită căutării în `/etc/hosts`, resolver-ul va căuta să acceseze un server DNS pentru a rezolva numele. Pe sistemele Unix, numele serverului DNS este uzual setat în fișierul `/etc/resolv.conf`.

Formatul `/etc/hosts` este simplu:

```
$ cat /etc/hosts
127.0.0.1      localhost
192.168.1.1    myserver
5.2.14.244     alex.unibuc.ro
```

Prima intrare din `/etc/hosts` ne indică așa-numita *adresa de loopback*. Aceasta este adresa locală a calculatorului și poate fi folosită ca orice adresă IP. Spre deosebire de o adresă IP din rețea, adresa de loopback poate fi folosită oricând, chiar și atunci când calculatorul nu este conectat fizic într-o rețea. În continuarea laboratorului veți folosi această adresă de loopback pentru a exersa toate comenzile care urmează, atunci când exercitiul nu indică expres o adresă de internet.

3 Servicii de rețea

Atunci când secvența de boot se încheie și kernelul ia controlul procesului `init` (`systemd` în Linux), dacă s-a selectat `runlevel`-ul care configurează sistemul ca fiind conectat în rețea (e.g., nivelurile 3 sau 5), am văzut la curs că se execută o serie de scripturi de sistem de tip *rc* sau *run commands* care pornesc și monitorizează serviciile sistem din spațiul utilizator. În particular, pentru `runlevel`-urile care prevăd funcționarea în rețea, aceste scripturi pornesc serviciile de rețea.

Serviciile de rețea în Linux se află în general în directorul `/etc/init.d` și sunt manipulate cu ajutorul comenzii `service`. Aceasta folosește numele serviciului (e.g., `ssh`) și comenzile pe care le înțelege scriptul aferent serviciului. În general aceste comenzi sunt standardizate pentru toate scripturile lansate de `init`: {start, stop, restart, reload, etc}. Iată o secvență de pornire/oprire a serviciului de `ssh`:

```
$ service ssh start
$ service ssh stop
```

Aici **ssh** este un script cu acest nume din `/etc/init.d` care poate primi ca parametru string-urile `{start, stop}`, executand pe cale de consecinta pornirea, respectiv oprirea serviciului. O secventa echivalenta de comenzi este:

```
$ /etc/init.d/ssh start
$ /etc/init.d/ssh stop
```

Serviciile de retea (si in general toate serviciile sistem) pot fi activate respectiv dezactivate cu comanda **systemctl** care controleaza activitatea procesului **systemd** si a managerului de servicii. Activarea si respectiv dezactivarea unui serviciu nu presupune pornirea sau oprirea lui, ci doar marcheaza serviciul ca fiind legat de o procedura anume din sistem, de pilda bootarea calculatorului. Activarea/dezactivarea si pornirea/oprirea serviciilor sunt ortogonale: un serviciu poate activat fara a fi pornit, dupa cum poate fi pornit fara a fi activat. Ca un exemplu concret, urmatoarea comanda:

```
$ systemctl disable ssh
```

nu va avea nici un efect asupra serviciului de **ssh**. Daca era pornit, el va continua sa ruleze. In schimb, la bootarea calculatorului serviciul nu va mai fi pornit automat, va fi nevoie de apelul explicit al comenzii **service** pentru a il porni.

N.B. Executia comenzilor **service** si **systemctl** necesita drepturi de *root*.

4 Acces la distanță

4.1 Transfer de date prin FTP

Pentru a accesa un *host* ce servește date prin protocolul FTP se folosește cumanda **ftp(1)**.

```
$ ftp alex@fmi.unibuc.ro
$ ftp ftp://fmi.unibuc.ro
$ ftp fmi.unibuc.ro -P 2121
```

Multe servere oferă informații legate de acces și structura datelor pe server la momentul conectării. Este important de văzut dacă este oferit acces anonim, fără autentificare. În acest caz de obicei se folosește utilizatorul **anonymous** și, politicos, se trece ca parola adresa de email la care puteți fi contactați. Dacă nu doriți acest lucru, apăsați pur și simplu **Enter** când se cere parola.

O dată conectați va apărea promptul **ftp>** care indică faptul că vă aflați într-un **shell** specializat protocolului FTP. Comenzile de navigare și manipulare a fișierelor (dacă aveți dreptul) sunt aceleași ca cele învățate până acum în **shell**: **ls**, **cd**, **pwd**, **rmdir**, **chmod** etc. Pentru a vedea toate comenzile disponibile apăsați la comanda **help**.

Pentru a urca sau coborî un fișier folosiți comenzile **put**, respectiv, **get**. Dacă aveți nevoie să efectuați operația pentru mai multe fișiere puteți folosi **mput** și **mget** (m de la *multiple*). Pentru a ieși folosiți **quit**.

Server-ele FTP sunt din ce în ce mai rare în spațiul public, dar comenzile și modul de lucru este comun cu înlocuitorii lor moderni (ex. **sftp**).

4.1.1 Instalarea și pornirea serviciului de FTP

Cel mai simplu mod de a exersa comanda **ftp** este să porniți local serviciul corespunzător (cel mai probabil instalându-l în prealabil):

```
$ apt install vsftpd
$ service vsftpd start
```

Obs: Dacă ați instalat software-ul cu comanda **apt**, nu este necesară pornirea serviciului, procedura de instalare a serverului îl și pornește automat. Pentru a verifica starea serviciului după instalare rulați comanda:

```
$ service vsftpd status
```

Odată ce serverul de FTP rulează, puteți folosi următoarea comandă:

```
$ ftp localhost
```

Puteți folosi la login contul *anonymous*? Dacă nu, de ce? (v. pagina de manual).

4.2 Administrare prin SSH

4.2.1 Instalarea și pornirea serviciului de SSH

La fel ca mai sus, începeți prin a porni serviciul de SSH (eventual instalându-l în prealabil dacă nu există în sistem):

```
$ service ssh status
$ service ssh start      # dacă serviciul e oprit
$ apt install openssh-server  # dacă serviciul nu e instalat
```

4.2.2 Lucrul cu SSH

Pentru a executa anumite comenzi sau a configura servicii de pe un *host* aflat la distanță se folosește comanda **ssh(1)**.

```
$ ssh fmi.unibuc.ro
$ ssh alex@fmi.unibuc.ro
$ ssh alex@fmi.unibuc.ro -p 2222
```

Rezultatul acestei comenzi este deschiderea unui shell pe o mașină aflată la distanță, identificată ca mai sus prin adresa de IP și/sau port. Numele de utilizator este fie implicit numele local al utilizatorului care lansează comanda **ssh**, fie cel precizat explicit în comandă înainte de caracterul **@**. Executia shell-ului la distanță esuează dacă utilizatorul nu reușește să se logheze în sistemul de la distanță cu parola de utilizator de pe sistemul respectiv.

O varianta mai comoda si mai sigura de autentificare, care nu presupune introducerea parolei de pe sistemul de la distanta, foloseste criptografia cu chei asimetrice. Aceasta metoda de autentificare presupune folosirea a doua chei: una *publica* si una *secreta/privata*. Cheia publica este cunoscuta tuturor (poate fi distribuita public) si poate fi preluata de sistemele de calcul care vor sa permita accesul pe baza ei. Cheia secreta este cunoscuta doar catre proprietarul contului si trebuie pastrata in siguranta. Compromiterea ei impune automat generarea unei noi perechi de chei si inlocuirea celor vechi. Perechea de chei este folosita pentru autentificare si acces fara parola. Cheile sunt stocate de regula in directorul `~/.ssh/` din contul utilizatorului. Cheia publica, care foloseste uzual extensia `pub` este distribuita dupa generare pe sistemele de calcul in care se doreste accesul utilizatorului. Ea este adaugata pe sistemele respective intr-un fisier numit `~/.ssh/authorized_keys` care contine toate cheile publice ale utilizatorului care are dreptul sa utilizeze contul respectiv de pe masina aflata la distanta.

La lansarea comenzii `ssh` se verifica continutul directorului `~/.ssh/` si intai se incarca autentificarea prin chei asimetrice, daca acestea exista in director. Altfel, `ssh` recurge la procedura de fall-back si se incearca metoda clasica de autentificare cu utilizator si parola.

O data autentificati, suntem intampinati de un `shell` identic cu cel cu care am lucrat pana acum doar ca ruleaza pe masina de la distanta, si ca atare toate comenzile sunt executate pe *host*-ul la distanta nu pe masina proprie.

Pentru a executa o simpla comanda fara a mai intra in `shell`, putem specifica comanda imediat dupa *host*:

```
$ ssh fmi.unibuc.ro ls
```

Generarea unei chei asimetrice se face cu comanda `ssh-keygen(1)`.

```
$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/alex/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/alex/.ssh/id_rsa.
Your public key has been saved in /home/alex/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:ixhRJajimfffiqTED8+ZGSp+ZMGtqwC/V7qsmxPTtAU alex@fmi
The key's randomart image is:
+----[RSA 2048]-----+
|      . o ..      |
|    o.Eo .        |
|. + o.o .         |
|...ooo.           |
|.  o++ S          |
|..++ooo...        |
|.  +*o = ....     |
|..o*@ * .         |
|.oBoX .           |
+-----+-----+
```

```
+-----[SHA256]-----+
```

Cheia publică are sufix `.pub` și se găsește în `/home/alex/.ssh/id_rsa.pub`. Cea privată se găsește în același loc dar fără sufix.

Implicit comanda `ssh-keygen(1)` generează chei `RSA`. Este recomandat să folosiți un algoritm mai nou cum ar fi `ed25519` sau `ecdsa`. Pentru aceasta folosiți argumentul `-t algorithm`:

```
$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
...
```

Cheile publice pentru cei care doriți să aibă acces pe contul dumneavoastră de pe un *host* (ex. calculatorul propriu, server web etc.) se pun în fișierul `.ssh/authorized_keys` din `$HOME`.

Pentru a adăuga o cheie publică `id_rsa.pub` folosiți

```
$ cat id_rsa.pub >> .ssh/authorized_keys
```

Pentru a transfera date prin `SSH` se folosește comanda `scp(1)` care se comportă aproape identic cu `cp(1)`. Diferența apare în specificarea sursei și destinației. Acestea sunt prefixate cu date legate de *host*.

```
$ scp hello.c fmi.unibuc.ro:
$ scp hello.c alex@fmi.unibuc.ro:code/
$ scp -r project/ alex@fmi.unibuc.ro:
```

Implicit, dacă nu este specificată nici o cale după `:`, transferul se face din/în directorul `$HOME` al utilizatorului. Dacă este specificată o cale, aceasta poate fi relativă `fmi.unibuc.ro:catalog` sau absolută `fmi.unibuc.ro:/etc/passwd`.

Adesea, comanda `scp` este folosită pentru a iniția transferuri de date de dimensiuni mari, care pot dura foarte mult, făcând impractică pastrarea deschisă a terminalului din care s-a lansat comanda. Pe de altă parte, închiderea terminalului echivalează în mod uzual cu terminarea comenzii, ceea ce nu este de dorit. Există programe care permit detasarea comenzii de terminalul de lucru, fapt ce permite închiderea acestuia. Ulterior, când utilizatorul reia sesiunea de lucru și deschide un nou terminal de lucru poate reatașa comanda noului terminal. În tot acest timp comanda a rulat în background și, presupunând că nu au existat erori în executia ei, a progresat în realizarea obiectivului ei. Programe de acest tip care detasează o comandă de terminalul de lucru sunt de pildă `screen` sau `tmux`. Comanda `scp` atunci când transferă cantități mari de date se folosește în mod uzual împreună cu o comandă care permite detasarea de terminal. Mai jos aveți un exemplu de folosire a comenzii `screen` care detasează de terminal o comandă `ssh` care execută la distanță o comandă care durează mult, simulată în exemplul nostru de `sleep 300`:

```
$ screen ssh localhost sleep 300
```

După autentificare (fără parolă dacă ați generat cu succes cheile asimetrice cf. procedurii de mai sus), comanda `ssh` va execută la distanță (în fapt local, pentru că v-ați conectat la *localhost*) comanda `sleep 300`. Apoi puteți tasta

Ctrl-a-d pentru a detasa comanda `ssh` de terminal si veti reprimi controlul shell-ului (promptul). Puteti inchide acum terminalul. Deschideti un terminal nou si executati comanda urmatoare:

```
$ screen -ls
```

care va lista un identificator al comenzii detasate de vechiul terminal. Cu ajutorul comenzii `screen -r <identificator>` puteti reatasta comanda `ssh` detasata anterior la terminalul curent. Tastati Ctrl-c pentru a termina executia comenzii `sleep`.

O implementare similară FTP folosind protocolul SSH este SFTP. Pentru a accesa un server se folosește comanda `sftp(1)` în același mod în care folosim comanda `ssh(1)`. O dată autentificați, comenzile și modul de lucru sunt aproape identice cu cele din FTP. Excepție face faptul că modul anonim nu mai este disponibil.

5 Sarcini de laborator

1. Găsiți adresele IP pentru `google.com`, `fmi.unibuc.ro`, `wikipedia.org`. Adăugați câte o intrare pentru fiecare în `/etc/hosts`.
2. Inter-schimbați adresele de IP pentru `google.com` și `fmi.unibuc.ro`. Folosiți `ping(1)` pentru cele două *host*-uri înainte și după modificare. Apare vreo schimbare?
3. Scrieti un shell script care citește tot conținutul fișierului `/etc/hosts` si pentru fiecare linie de tip adresă IP – nume folosiți comanda `nslookup` pentru a verifica adresa de IP a numelor din fișier. Dacă `nslookup` va întoarce o altă adresă IP decât cea din `/etc/hosts` tipăriți pe ecran mesajul "Bogus IP for <nume> in /etc/hosts !".
Indicație: O posibilă soluție este să folosiți comenzile `cat` și `while` într-un pipeline, `cat` pentru a afișa conținutul `/etc/hosts` și `while` împreună cu `read` pentru a itera prin conținutul fișierului.
4. Inspectați cu un program de tip pager (`less/more`) conținutul scriptului `/etc/init.d/ssh`. Înțelegeți felul în care sunt folosiți parametrii de apel `start|stop|status|reload`, etc.?
5. Accesați serverul FTP `ftp.vim.org` folosind un client `ftp` din linie de comandă. Navigați în directorul `pub/vim/pc/` și obțineți fișierul `vimXXsrc.zip` unde `XX` este cea mai recentă versiune pe care o găsiți în acel director (indicativ: folosiți `ls`).
6. Inspirați-vă din ghidul <https://cloud.google.com/compute/docs/tutorials/basic-webserver-apache> pentru a vă crea pe mașina locală o mașină virtuală Linux care servește pagini de Web. Porniți serviciul de `ssh` din mașina virtuală, eventual instalând serviciul în prealabil dacă nu există pe mașina virtuală. Folosiți `ssh-keygen(1)` pentru a genera o pereche

de chei publica-privata. Copiați cheia publică (cea cu extensia `.pub`) pe contul pe care îl folosiți pentru a accesa mașina virtuală și adăugați-o la conținutul fișierului `authorized_keys` ca mai sus. Conectați-vă prin `ssh(1)` la noua mașină virtuală de pe calculatorul local.

7. Copiați un mic site Web de pe mașina locală pe care o folosiți pe serverul Web de pe mașina virtuală creată anterior. Acest task revine la a copia directorul care conține site-ul Web pe mașina locală pe mașina virtuală (server) folosind `scp(1)`. Noul director trebuie pus în `/var/www/html/student/`.
Indicații: Pentru Windows puteți transfera fișierele folosind comanda `scp` dintr-un shell `cygwin` sau cu ajutorul `putty`.
Pentru a intra rapid în posesia unui mini site web, puteți downloada de pe internet cu comanda `wget` codul html al unui site existent. Folosiți flag-urile `-r` (download recursiv) și `-l` pentru a limita adâncimea arborelui de documente html descărcat. Ex: `wget -r -l 2 fmi.unibuc.ro`

Ubuntu [Running]

Nov 18 10:57

fernando@fernando-VirtualBox: ~/Desktop

 fernando@fernando-VirtualBox:~/Desktop\$ nslookup google.com

Server: 127.0.0.53

Address: 127.0.0.53#53

 Non-authoritative answer:

Name: google.com

Address: 172.217.17.110

 Name: google.com

Address: 2a00:1450:4017:810::200e

 fernando@fernando-VirtualBox:~/Desktop\$ nslookup fmi.unibuc.ro

Server: 127.0.0.53

Address: 127.0.0.53#53

 Non-authoritative answer:

Name: fmi.unibuc.ro

Address: 80.96.21.88

 fernando@fernando-VirtualBox:~/Desktop\$ nslookup wikipedia.org

Server: 127.0.0.53

Address: 127.0.0.53#53

Non-authoritative answer:

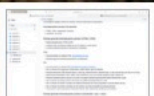
Name: wikipedia.org

Address: 185.15.58.224

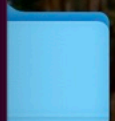
Name: wikipedia.org

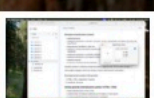
Address: 2a02:ec80:600:ed1a::1

fernando@fernando-VirtualBox:~/Desktop\$

 Screenshot

2024-11...14.11.32

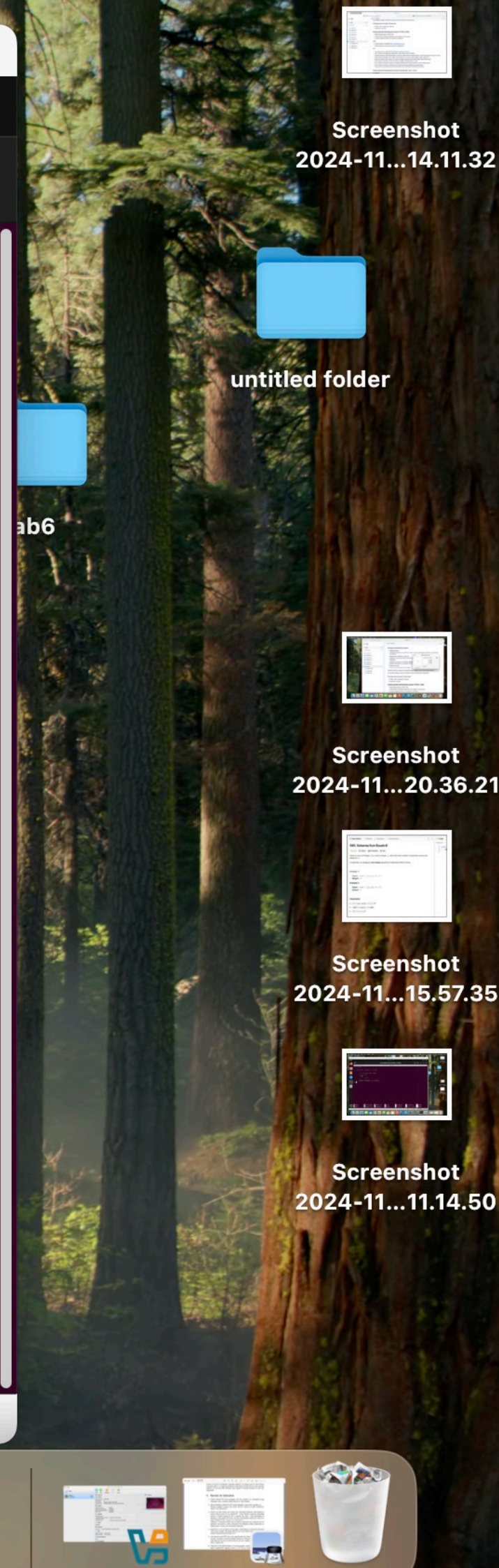
 ed folder

 Screenshot

2024-11...20.36.21

 Screenshot

2024-11...15.57.35



Ubuntu [Running]

Nov 18 11:22

root@fernando-VirtualBox: /etc/init.d

```
#!/bin/sh
### BEGIN INIT INFO
# Provides:          sssd
# Required-Start:    $remote_fs $syslog
# Required-Stop:     $remote_fs $syslog
# Should-Start:      $named
# Default-Start:     2 3 4 5
# Default-Stop:      0 1 6
# Short-Description: System Security Services Daemon
# Description:       Provides a set of daemons to manage access to
#                   remote directories and authentication
#                   mechanisms. It provides an NSS and PAM interface
#                   toward the system and a pluggable backend system
#                   to connect to multiple different account sources.
### END INIT INFO
# start on filesystem
# stop on runlevel [06]

DESCRIPTION="System Security Services Daemon"
PATH=/bin:/usr/bin:/sbin:/usr/sbin
NAME=sssd
DAEMON_OPTS=""
DAEMON=/usr/sbin/$NAME
PIDFILE=/var/run/$NAME.pid

# Load the VERBOSE setting and other rcS variables
. /lib/init/vars.sh

--More-- (35%)
```

Screenshot
2024-11...14.11.32

untitled folder

ab6

Screenshot
2024-11...20.36.21

Screenshot
2024-11...15.57.35

Ubuntu [Running]

Nov 18 11:03

fernando@fernando-VirtualBox: ~/Desktop

GNU nano 7.2 /etc/hosts *

127.0.0.1 localhost
127.0.1.1 fernando-VirtualBox

142.250.184.142 fmi.unibuc.ro
80.96.21.88 google.com
185.15.59.224 wikipedia.org

The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
|

Help

Exit

Write Out

Read File

Where Is

Replace

Cut

Paste

Execute

Justify

Location

Go To Line

M-U

M-E

Undo

Redo

Left

Screenshot 2024-11...14.11.32

ed folder

Screenshot 2024-11...20.36.21

Screenshot 2024-11...15.57.35

ex 1

Ubuntu [Running]

Nov 18 11:06

fernando@fernando-VirtualBox: ~/Desktop

fernando@fernando-VirtualBox:~/Desktop\$ ping google.com

PING google.com (80.96.21.88) 56(84) bytes of data.

64 bytes from google.com (80.96.21.88): icmp_seq=1 ttl=255 time=17.9 ms

64 bytes from google.com (80.96.21.88): icmp_seq=2 ttl=255 time=15.5 ms

64 bytes from google.com (80.96.21.88): icmp_seq=3 ttl=255 time=15.8 ms

64 bytes from google.com (80.96.21.88): icmp_seq=4 ttl=255 time=13.0 ms

64 bytes from google.com (80.96.21.88): icmp_seq=5 ttl=255 time=15.2 ms

64 bytes from google.com (80.96.21.88): icmp_seq=6 ttl=255 time=17.8 ms

64 bytes from google.com (80.96.21.88): icmp_seq=7 ttl=255 time=16.8 ms

^C

--- google.com ping statistics ---

7 packets transmitted, 7 received, 0% packet loss, time 6005ms

rtt min/avg/max/mdev = 12.967/16.000/17.910/1.588 ms

fernando@fernando-VirtualBox:~/Desktop\$ ping fmi.unibuc.ro

PING fmi.unibuc.ro (142.250.184.142) 56(84) bytes of data.

64 bytes from fmi.unibuc.ro (142.250.184.142): icmp_seq=1 ttl=255 time=76.7 ms

64 bytes from fmi.unibuc.ro (142.250.184.142): icmp_seq=2 ttl=255 time=55.9 ms

64 bytes from fmi.unibuc.ro (142.250.184.142): icmp_seq=3 ttl=255 time=49.6 ms

^C

--- fmi.unibuc.ro ping statistics ---

4 packets transmitted, 3 received, 25% packet loss, time 3005ms

rtt min/avg/max/mdev = 49.625/60.753/76.733/11.586 ms

fernando@fernando-VirtualBox:~/Desktop\$

Screenshot
2024-11...14.11.32

untitled folder

Screenshot
2024-11...20.36.21

Screenshot
2024-11...15.57.35

ex 1

Screenshot
2024-11...11.03.22

Ubuntu [Running]

Nov 18 11:09

fernando@fernando-VirtualBox: ~/Desktop

GNU nano 7.2 /etc/hosts *

127.0.0.1 localhost
127.0.1.1 fernando-VirtualBox

80.96.21.88 fmi.unibuc.ro
142.250.184.142 google.com
185.15.59.224 wikipedia.org

The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

Help
Exit

Write Out
Read File

Where Is
Replace

Cut
Paste

Execute
Justify

Location
Go To Line

M-U Undo
M-E Redo

Left ⌘

Screenshot 2024-11...14.11.32

untitled folder

Screenshot 2024-11...20.36.21

Screenshot 2024-11...15.57.35

ex 1_1

ex1_2

Ubuntu [Running]

Nov 18 11:10

fernando@fernando-VirtualBox: ~/Desktop

```
fernando@fernando-VirtualBox:~/Desktop$ ping google.com
PING google.com (142.250.184.142) 56(84) bytes of data.
64 bytes from google.com (142.250.184.142): icmp_seq=1 ttl=255 time=56.0 ms
64 bytes from google.com (142.250.184.142): icmp_seq=2 ttl=255 time=55.6 ms
64 bytes from google.com (142.250.184.142): icmp_seq=3 ttl=255 time=54.5 ms
^C
--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 54.459/55.329/55.969/0.637 ms
fernando@fernando-VirtualBox:~/Desktop$ ping fmi.unibuc.ro
PING fmi.unibuc.ro (80.96.21.88) 56(84) bytes of data.
64 bytes from fmi.unibuc.ro (80.96.21.88): icmp_seq=1 ttl=255 time=29.1 ms
64 bytes from fmi.unibuc.ro (80.96.21.88): icmp_seq=2 ttl=255 time=12.5 ms
64 bytes from fmi.unibuc.ro (80.96.21.88): icmp_seq=3 ttl=255 time=15.3 ms
64 bytes from fmi.unibuc.ro (80.96.21.88): icmp_seq=4 ttl=255 time=17.9 ms
^C
--- fmi.unibuc.ro ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 12.493/18.705/29.106/6.306 ms
fernando@fernando-VirtualBox:~/Desktop$
```

Screenshot 2024-11...14.11.32

untitled folder

Screenshot 2024-11...20.36.21

Screenshot 2024-11...15.57.35

ex 1_1

ex1_2

Ubuntu [Running]

Nov 18 18:47

fernando@fernando-VirtualBox: ~/Desktop/lab7

fernando@fernando-VirtualBox: ~/Desktop/lab7v2

fernando@fernando-VirtualBox: ~/Desktop/lab7

```
* .
. o = o
=.oE + S .
o+=oo.o .
++0o=+.+.
|=+o=+Bo+ .
|oooo=+o.o
+----[SHA256]-----+
fernando@fernando-VirtualBox:~/Desktop/lab7$ cat ~/.ssh/id_ed25519
cat: invalid option -- '/'
Try 'cat --help' for more information.
fernando@fernando-VirtualBox:~/Desktop/lab7$ cat ~/.ssh/id_ed25519
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABG5vbmUAAAABbm9uZQAAAAAAAAABAAAAMwAAAAAtzc2gtZW
QyNTUxOQAAACBjbE0zlgvmO42Sh7UofQkSyiDuum3CGobvSmnDm09gBgAAAKAVuioTFboq
EwAAAAAtzc2gtZWQyNTUxOQAAACBjbE0zlgvmO42Sh7UofQkSyiDuum3CGobvSmnDm09gBg
AAAEAZUKMH2qw/BmvVH5JeivJM1AGZqZTZn73HkrTA/+g4CUlsQ70WC+Y7jZKhtSh9CRLK
IO66bcIahu9KacObT2AGAAAAGWZlcm5hbmRvZG9uZWExMEBnbWVpY20BAGME
-----END OPENSSH PRIVATE KEY-----
fernando@fernando-VirtualBox:~/Desktop/lab7$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIElsQ70WC+Y7jZKhtSh9CRLKIO66bcIahu9KacObT2AG fernandodonea10@gmail.com
fernando@fernando-VirtualBox:~/Desktop/lab7$ ssh -T git@github.com
The authenticity of host 'github.com (140.82.121.3)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
git@github.com: Permission denied (publickey).
fernando@fernando-VirtualBox:~/Desktop/lab7$ cat
```

Left ⌘

Applications

Documents

Music

FinderAppStoreCalendarNotesRemindersPhotosPhotosLibraryMessagesMailKeychainAccessoriesSystemPreferences

VSCodeTerminalScreenshot 24-11...18.21.02Screenshot 24-11...15.57.35Screenshot 24-11...20.36.21Screenshot 24-11...14.11.32

Trash